Enoncé du TP : Jeu Vidéo en Python avec MongoDB

Objectif

Développer un jeu en ligne de commande permettant aux joueurs de composer des équipes de personnages pour affronter des monstres en combat infini. Le jeu utilise MongoDB pour stocker les données.

Fonctionnalités requises

1. **Base de données MongoDB**
    - Stockage de 10 personnages jouables avec des caractéristiques prédéfinies (attaque, défense, points de vie).
    - Stockage de plusieurs types de monstres.
    - Historique des 3 meilleurs scores avec le nom d'utilisateur du joueur.
2. **Création d'équipe**
    - Le joueur peut composer une équipe de 3 personnages parmi ceux disponibles.
    - Les informations des personnages doivent être récupérées depuis la base de données.
3. **Système de combat**
    - Une fois l'équipe créée, le joueur affronte des monstres aléatoires tirés de la base de données.
    - Le combat se poursuit à l'infini, avec un compteur de vagues qui s'incrémente à chaque victoire.
    - Chaque personnage possède des points de vie (PV), s'ils atteignent zéro, l'équipe est vaincue.
4. **Classement des meilleurs scores**

- À la fin du jeu, le score du joueur (nombre de vagues survécues) est enregistré.
- Les 3 meilleurs scores doivent être affichés.

Fonctionnement d'un tour de jeu

1. **Sélection aléatoire d'un monstre**
   - Un monstre est tiré au hasard depuis la base de données.
   - Ses caractéristiques (attaque, défense, points de vie) sont affichées.
2. **Phase de combat**
   - Chaque personnage de l'équipe attaque le monstre et réduit ses PV en fonction de l'attaque et de la défense.
   - Le monstre attaque un personnage aléatoire de l'équipe et inflige des dégâts.
   - La défense du personnage ciblé réduit les dégâts infligés d'un montant équivalent à sa valeur de défense. (Fonctionne aussi sur les monstres)
   - Si les PV du monstre atteignent zéro → **Victoire**.
   - Si tous les personnages de l'équipe atteignent zéro PV → **Défaite**, fin de la partie.
   - Les personnages de l'équipe attaquent avant le monstre.
3. **Mise à jour du compteur de vagues**
   - Si le joueur gagne, le compteur de vagues est incrémenté de 1.
   - Une nouvelle vague est lancée avec un nouveau monstre aléatoire.
4. **Stockage des résultats**
   - Après la fin de la partie (défaite), le nombre de vagues survécues est sauvegardé dans la base de données.
5. **Affichage des résultats**
   - Après la fin de la partie, le joueur voit son score ainsi que le classement des meilleurs scores enregistrés.

Fonctionnalités de l'application en ligne de commande

1. **Interaction avec l'utilisateur**
    - Menu principal proposant les options suivantes :
        1. Démarrer le jeu
        2. Afficher le classement
        3. Quitter
    - Apres demarrer le jeu:
        - Choix nom utilisateur
        - Creation de l'equipe
    - Menu de creation de l'equipe:
        - Affichage de l'equipe actuel
            1. Personnage numero 1 - X ATK - X DEF - X PV
            2. Personnage numero 2 - X ATK - X DEF - X PV
            3. ....
2. **Gestion des équipes**
    - Saisie du nom du joueur avant la création d'une équipe.
    - Sélection des personnages pour constituer une équipe, chaque personnage ne peut être sélectionné qu'une seule fois.
    - L'interface propose une liste des personnages disponibles avec leurs caractéristiques.
3. **Combat automatisé**
    - Affrontement contre des monstres en boucle jusqu'à la défaite.
    - Mise à jour du score et sauvegarde dans la base de données.

Critères de réussite

- Le jeu doit fonctionner sans plantage et offrir une interaction fluide. (Attention au entrée utilisateurs)

- Les données doivent être correctement stockées et récupérées depuis MongoDB.
- Le classement doit refléter les trois meilleurs scores.

Instructions de mise en œuvre

Prérequis

- Installer Python (>= 3.8)
- Installer MongoDB
- Bibliothèques Python requises :

```
pip install pymongo
```

Exemple de structure du projet

```
jeu_video_python/
|-- main.py
|-- db_init.py
|-- game.py
|-- utils.py
|-- models.py
|-- README.md
```

Implémentation

1. **Initialisation de la base de données**
   - Script `db_init.py` pour insérer les personnages et monstres dans MongoDB.
2. **Lancement du jeu**
   - Script `main.py` proposant le menu principal et gérant les interactions.
3. **Gestion des combats**
   - Script `game.py` gérant la logique du combat.

4. **Modélisation des données**
    - Script `models.py` contenant les classes pour les personnages et les monstres.
5. **Fonctions utilitaires**
    - Script `utils.py` contenant les fonctions pour l'affichage et la gestion des données.

Exécution du projet

1. Initialiser la base de données :

```
python db_init.py
```

2. Lancer le jeu :

```
python main.py
```

Annexe

Exemple de personnages :

Guerrier - ATK: 15, DEF: 10, PV: 100

Mage - ATK: 20, DEF: 5, PV: 80

Archer - ATK: 18, DEF: 7, PV: 90

Voleur - ATK: 22, DEF: 8, PV: 85

Paladin - ATK: 14, DEF: 12, PV: 110

Sorcier - ATK: 25, DEF: 3, PV: 70

Chevalier - ATK: 17, DEF: 15, PV: 120

Moine - ATK: 19, DEF: 9, PV: 95

Berserker - ATK: 23, DEF: 6, PV: 105

Chasseur - ATK: 16, DEF: 11, PV: 100

Exemple de monstres :

Gobelin - ATK: 10, DEF: 5, PV: 50

Orc - ATK: 20, DEF: 8, PV: 120

Dragon - ATK: 35, DEF: 20, PV: 300

Zombie - ATK: 12, DEF: 6, PV: 70

Troll - ATK: 25, DEF: 15, PV: 200

Spectre - ATK: 18, DEF: 10, PV: 100

Golem - ATK: 30, DEF: 25, PV: 250

Vampire - ATK: 22, DEF: 12, PV: 150

Loup-garou - ATK: 28, DEF: 18, PV: 180

Squelette - ATK: 15, DEF: 7, PV: 90